Ders 1 Genişletilmiş Notu: Algoritma Tasarım İlkeleri, Doğrusal Arama, Eratosthenes Kalburu ve Sıralı Dizilerin Birleştirilmesi

Ders 1 Genişletilmiş Notu: Algoritma Tasarım İlkeleri, Doğrusal Arama, Eratosthenes Kalburu ve Sıralı Dizilerin Birleştirilmesi

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

Kayıtta dersin iki bölümü ters sırada birleşmiştir. Bu not gerçek ders sırasını izler: önce dersin tanıtımı, sonra teknik anlatım.

İçindekiler

- 1. Veri Yapıları ve Algoritmalar Dersinin Amacı
- 2. Gerçek Hayattan Veri Yapısı Örnekleri
- 3. Dersin Konu Planı ve Kaynaklar
- 4. Sırasız Dizide Doğrusal Arama
- 5. Bir Algoritma Nasıl Tasarlanır?
- 6. İyi Bir Algoritmanın Ölçütleri
- 7. Eratosthenes Kalburu ile Asal Sayıların Bulunması
- 8. İki Sıralı Dizinin Birleştirilmesi
- 9. Düşünme Sorusu: Hash Tablosunda Ortalama Yerleşme Adımı

1. Veri Yapıları ve Algoritmalar Dersinin Amacı

Birinci sınıfta ve ikinci sınıfın ilk döneminde temel kavramlar görüldü: bir algoritma nasıl tasarlanır, karar verme nasıl yapılır, birden fazla kez yapılacak bir iş için döngü nasıl kullanılır, bunlar C dilinde nasıl kodlanır. Hocaya göre başlangıçta C'nin seçilmesinin nedeni, temel bir programlama dilini iyi öğretmektir. Giriş ve çıkış, kontrol yapıları, dosya kullanımı ve dinamik bellek kullanımı öğrenilirse diğer dillerde yalnızca o dile özgü farklar kalır. Bir programlama dilinin neler yapabileceğini bilen kişi diğer dilleri kendi başına da kolayca öğrenir. Bu derste de kod yazmak gerektiğinde C kullanılır.

1.1 Algoritmalar Kısmı

Çok kullanılan bazı algoritmalar vardır. Bunların başında sıralama algoritmaları gelir, çünkü bilgiye sıralı erişim zaman kazandırır. Hocanın örneği cep telefonundaki kişiler listesidir. Yüzlerce kişinin bilgisi harf sırası olmadan dursa, gözle arayarak bir kişiyi bulmak için çok zaman kaybedilirdi. Liste sıralı olduğunda B harfine gelip oradan ilerlemek yeter. Bilgisayar da benzer bir yol izler.

Bilgi aslında erişmek için sıralanır. Bu yüzden ikinci grup arama algoritmalarıdır: bilgi sırasızsa nasıl erişilir, sıralıysa nasıl daha verimli erişilir? Kişi adlarında konu sayıların ötesine geçer. "Berna, Banu'dan sonra gelmeli" gibi bir sıralama metinler üzerinde yapılır. Bir de metin arama (string search) vardır. Editörde "bu dokümanda şu kelime neredeydi?" diye aradığımızda sonucu en hızlı şekilde gösterecek bir algoritma gerekir.

Hoca verimliliğe (efficiency) neden bu kadar önem verildiğini kullanıcı davranışıyla açıkladı. Bugün işlerin çoğu bilgisayarla yapılıyor. Kullanılan bir araç biraz yavaş ya da sıkıntılı olsa kullanıcı hemen başka bir araca geçmek isteyebilir.

Algoritmaları öğrenmenin iki faydası vardır. Birincisi, quicksort veya mergesort gibi bir algoritmayı bilen kişi gerektiğinde onu hazır kullanır. İkincisi ve daha önemlisi, algoritmalar düşünme şekli öğretir. "Quicksort'ta şöyle bir tasarım yapıyorduk, ben bunu şuradaki problem için değiştirerek kullanabilirim" denebilir. Bir metin arama algoritmasının mantığı başka bir probleme uygulanabilir.

> **Hocanın vurgusu:** Bu algoritmaların faydası yalnızca Boyer-Moore'u ya da quicksort'u bilmek değildir. Önemli olan, öğrenilen tasarım tekniklerini kendi tasarlayacağınız bambaşka bir algoritmada kullanabilmektir.

1.2 Veri Yapıları Kısmı

Şimdiye kadar görülen veri yapıları dizi ve matristir. Bazı problemler ise o problemin çözümüne özgü veri yapıları ister. Dersin ilk hedefi, ana veri yapılarını ve bunlar için kullanılan temel algoritmaları, ayrıca sık kullanılan önemli algoritmaları öğretmektir. Bunu yaparken olabildiğince verimli tasarımlar yapmaya yönlendirmek de hedeftir.

2. Gerçek Hayattan Veri Yapısı Örnekleri

Hoca üç veri yapısını güncel örneklerle tanıttı. Dikkat çektiği nokta şuydu: Bu yapılar genel problem türleri için geliştirilmiştir. Bu yüzden birbirine hiç benzemeyen iki uygulama aynı veri yapısını kullanabilir.

2.1 Graf

Harita uygulamasında "Kadıköy'deyim, Davutpaşa'ya gideceğim, bana uygun yolları göster" diyebilirsiniz. Uygulama bir yol (path) çizer. Yol üzerinde Beşiktaş, Zincirlikuyu, Mecidiyeköy gibi uğranan noktalar vardır. Kadıköy'den Beşiktaş'a geldikten sonra oradan başka bir yolla da Davutpaşa'ya gidilebilir. Bu yüzden uygulamalar birden fazla yol gösterir.

Böyle bir problem için uygun yapı graf (graph) veri yapısıdır. Düğümler (nodes) semtlerdir. Düğümler arasındaki bağlantılar (edges), iki nokta arasındaki kilometre veya o anda kaç dakika sürdüğü gibi bilgileri taşır.

İkinci örnek sosyal ağlardır. Hocaya göre Facebook, Instagram ve Twitter gibi programlar da graf veri yapısını kullanır. Düğümler kişileri, bağlantılar ise kişiler arasındaki ilişkiyi (kardeş, arkadaş, eş gibi) gösterir. Harita ile sosyal ağ çok farklı problemlerdir ama ikisinin mantığı da aynıdır: düğümler, bağlantılar ve bağlantıların ifade ettiği bilgi.

2.2 Yığın

Bir web sitesinde sırayla Yıldız'ın ana sayfasına, akademik birimlere, oradan Elektrik-Elektronik Fakültesi sayfasına geçtiğinizi düşünün. "Geri" tuşuna bastığınızda akademik birimlere dönersiniz, doğrudan ana sayfaya atlamazsınız. Her "geri" sizi bir önceki noktaya götürür. Bu iş için yığın (stack) veri yapısı kullanılır. Yığın, son giren ilk çıkar (LIFO, last in first out) mantığıyla çalışır.

Aynı mantık fonksiyon çağrılarında da vardır. Ana programdan `A` fonksiyonu, `A`'dan `B`, `B`'den `C` çağrıldıysa, `C` bitince doğrudan ana programa dönülmez. Önce `B`'ye, `B` bitince `A`'ya, `A` bitince ana programa dönülür.

> **Ders sonrası not:** Hoca bu işi derleyicinin yaptığını söyledi; doğrusu, dönüş bilgisi program çalışırken çağrı yığınında tutulur. Derleyici bunu yapacak kodu üretir.

2.3 Kuyruk

Bir ağ üzerindeki yazıcıya herkes çıktı gönderiyor olsun. İstekler kuyruğa girer ve sırayla hizmet verilir. Bu, ilk giren ilk çıkar (FIFO, first in first out) mantığıyla çalışan kuyruk (queue) veri yapısıdır. Önce birinci istek biter, sonra ikincisi, sonra üçüncüsü.

Derste bu yapıların hem nasıl tasarlanacağı hem de nasıl kullanılacağı görülecektir.

3. Dersin Konu Planı ve Kaynaklar

Dönem boyunca planlanan konular sırasıyla şunlardır:

- Temel algoritma tasarımı örnekleri.
- Algoritma analizine giriş: en iyi, en kötü ve ortalama durum karmaşıklığı nasıl hesaplanır. Ayrıntılar üçüncü sınıftaki Algoritma Analizi dersindedir.
- Liste (dizi, matris) ve bağlı listeler (linked lists).
- Kuyruk ve yığın, örnek algoritmalarla.
- Ağaçlar: ikili ağaçlar (binary trees), ikili arama ağaçları (binary search trees).
- Heap veri yapısı ve heap üzerinde sıralama.
- Öncelikli kuyruk (priority queue).
- Graflar, minimum kapsayan ağaç (minimum spanning tree) ve graf kullanımları.
- Böl ve yönet (divide and conquer) algoritmaları: quicksort, mergesort.
- Metin arama algoritmaları.
- Sayarak sıralama gibi diğer sıralama algoritmaları.

Soru: Bağlı liste ve dinamik bellek önceki derslerde görüldü, burada yeniden anlatılacak mı? **Cevap:** Ağaç ve graf gibi yapılar bağlı listeyi kullandığı için bağlı liste sıfırdan anlatılacak.

Soru: Ağaç ve graf ayrık matematikte görülmedi mi? **Cevap:** Ayrık matematikte bunlar yapı olarak anlatılır. Bu derste ise tasarımları ve algoritmalarda nasıl kullanılacakları anlatılır.

Soru: Ders kaynağındaki kitaplar hakkında bilgi verilebilir mi? **Cevap:** Hoca iki kitap önerdi. Birincisi Levitin'in algoritma tasarımı ve analizi kitabıdır. Hoca bu kitabı en sevdiği kitap olarak tanıttı, derste ondan çok yararlandığını söyledi. İkincisi Princeton'dan Sedgewick'in *Algorithms* kitabının dördüncü baskısıdır. Üçüncü sınıftaki Algoritma Analizi dersinde de bu iki kitaptan devam edilir. Hocaya göre ikisi de konuları çok temelden anlatan, ileride kitaplıkta tutmaya değer kitaplardır.

> **Ders sonrası not:** Hoca birinci kitabın adını "Design and Analysis of Algorithms" diye söyledi; tam adı büyük olasılıkla Anany Levitin, *Introduction to the Design and Analysis of Algorithms*'tir. İkinci kitabın ortak yazarı büyük olasılıkla Kevin Wayne'dir.

4. Sırasız Dizide Doğrusal Arama

Hoca teknik kısma bilerek çok basit bir algoritmayla başladı: sırasız bir dizide bir elemanın yerini bulmak, yani doğrusal arama (linear search). Amacı bir dönem boyunca algoritma tasarlarken nelere dikkat edileceğini göstermekti. Bu hataları üçüncü sınıfta bile sınav kağıtlarında gördüklerini söyledi. Bu yüzden konunun üzerinden yavaşça geçti.

Örnek dizi şudur:

$$A = \{4,\ 5,\ 2,\ 1,\ 7,\ 9\}, \qquad n = 6$$

Önce problem iyi anlaşılmalıdır. Sonra basit bir örnek hazırlanır ve bu örnekte üç şey değerlendirilir: genel durum, uç noktalar ve özel durum.

4.1 Genel Durum ve Adım Sayısı

$x = 7$ olsun. Dizinin başından başlanır: 4, 5, 2 ve 1 ile karşılaştırma başarısız olur. Beşinci karşılaştırmada, 4 numaralı indiste 7 bulunur ve durulur.

Bu noktada hoca bir soru sordu: Burada adım sayısı belli olan bir döngü mü kullanılmalı, belli olmayan bir döngü mü? Adım sayısı belli değildir. $x = 7$ için 4 numaralı indiste durulur, ama $x = 5$ deseydik aranan eleman 1 numaralı indiste karşımıza çıkardı. Eleman bulunduğu anda işlem bitmelidir.

4.2 Yanlış Tasarım: Döngüyü Kırmak

Pek çok öğrenci bunu adım sayısı belli bir döngüyle yazıp şart sağlanınca döngüyü kırarak çözer:

```
for (i = 0; i < n; i++) {
    if (A[i] == x)
        break;
}
```

Bu kod çalışır, ama tasarım olarak yanlıştır. Eleman bulununca çıkılması gerektiği, algoritmayı tasarlarken zaten bilinir. Bu bir sürpriz değildir. Adım sayısı belli olmayan bir işi, adım sayısı belliymiş gibi kurup şart bozulunca kırmak hatalı tasarımdır.

> **Hocanın vurgusu:** Döngüleri kırarak çıkmaya çalışmak çok yapılan bir hatadır. İkinci sınıfın ikinci döneminde artık yapılmamalıdır. Algoritmanın çalışıyor olması yeterli değildir; temel tasarım kurallarına ve verimlilik şartlarına da uymalıdır.

4.3 Özel Durum: Eleman Dizide Yoksa

Aranan eleman dizide yoksa ne olur? $x = 11$ olsun. Her elemana bakılır, dizinin sonuna gelinir ve dizi bitince iş bitmiş olmalıdır. İndisler 0'dan $n - 1$'e kadar gittiğine göre döngü şöyle kurulur: $i < n$ olduğu ve $A[i] \neq x$ olduğu sürece i bir artırılır.

4.4 Koşulların Sırası

Hoca şu soruyu sordu: $A[i] \neq x$ koşulunu önce yazsak, $i < n$ koşulunu sonra yazsak bir şey fark eder mi?

Soru: Koşulların yeri değişirse fark eder mi? **Cevap:** Bir öğrenci aradaki bağlaç "ve" olduğu için fark etmeyeceğini söyledi. Hoca soruyu $x = 11$ için yineledi. Başka bir öğrenci doğru cevabı verdi: dizinin boyutu aşılır, dizide olmayan bir yer kontrol edilmeye çalışılır.

Bunu $x = 11$ için adım adım görelim. Önce $i < n$ yazılan doğru kurulumda şunlar olur:

i	$i < 6$ mı?	$A[i]$	$A[i] \neq 11$ mi?	Sonuç
0	Evet	4	Evet	i artar
1	Evet	5	Evet	i artar
2	Evet	2	Evet	i artar
3	Evet	1	Evet	i artar
4	Evet	7	Evet	i artar
5	Evet	9	Evet	i artar
6	Hayır	bakılmaz	bakılmaz	Döngüden çıkılır

Koşullar ters yazılsaydı $i = 6$ için önce $A[6] \neq x$ sorulurdu. Oysa $A[6]$ diye bir eleman yoktur. Hoca iki durumu ayırdı:

- **Dizi `int A[6]` olarak tanımlıysa:** geçerli indisler 0-5'tir. $A[6]$ size ait olmayan bir gözdür.
- **Dizi `int A[50]` olarak tanımlanıp 6 eleman okunduysa:** $A[6]$ size aittir ama siz oraya eleman yerleştirmediniz. Orada rastgele değer (garbage) vardır.

Hocanın benzetmesi şuydu: Uzaya füze attığınızı, çok hassas bir iş yaptığınızı düşünün. Milyarda bir ihtimalle o gözde tesadüfen tam aradığınız sayı vardır. Program elemanı bulduğunu sanar ve füzenin başına bir iş gelir. Orası bellekte gerçekten var olan bir bölgedir ve içinde bir değer vardır. Oraya yazmıyorsunuz, bir şey bozmuyorsunuz, ama arama yanlış sonuç verebilir.

> **Ders sonrası not:** Hoca `A[50]` örneğinde "5 eleman için okuma yapmışsınız" dedi, hemen ardından 6 eleman olduğunu söyledi. Doğrusu 6 elemandır; 6 numaralı göz bu yüzden boştur.
>
> **Ek bilgi (derste anlatılmadı):** Bu sıralamanın işe yaramasının nedeni C'deki kısa devre değerlendirmesidir (short-circuit evaluation). `p && q` ifadesinde `p` yanlışsa `q` hiç hesaplanmaz. Bu yüzden $i = 6$ olduğunda `A[6]` okunmaz. Bir öğrenci derste "ilk

kontrol sağlanıyorsa ikincisi yapılmaz” dedi; && için doğrusu tersidir, ikinci koşul ilk koşul yanlışsa atlanır.

Hocanın vurgusu: Döngü şartında dizi üzerinde kontrol yapıyorsanız ve dizinin bitme ihtimali varsa, önce dizinin bitip bitmediğini kontrol edin. Dizi ile ilgili işlemi ondan sonra yapın.

4.5 Döngüden Çıkınca Kontrol

Döngüden iki nedenle çıkılabilir: eleman bulundu ya da dizi bitti. Bu yüzden sonucu yazdırmadan önce neden çıkıldığına bakılmalıdır. $i = n$ ise eleman dizide yoktur, değilse eleman i indisindedir. Bu kod bir fonksiyon olarak yazılsaydı, bulunan yeri ya da bulunamadığını döndürürdü. Çağıran tarafın da dönen değeri kontrol etmesi gerekirdi. Tasarımın tamamı şöyledir:

```
N, A[N] oku
x oku
i = 0
(i < N) ve (A[i] != x) olduğu sürece:
    i = i + 1
eğer i == N ise:
    "YOK" yazdır
değilse:
    i yazdır
```

Derste kod değil akış diyagramı (flowchart) yazıldı. Aşağıdaki C kodu tasarımın karşılığıdır, hocanın fonksiyon önerisine göre yazılmıştır, `gcc -Wall` ile derlenip çalıştırılmıştır.

```c
#include <stdio.h>

/* Bulursa indisi, bulamazsa -1 döndürür. */
int dogrusal_ara(const int a[], int n, int x)
{
    int i = 0;

    while (i < n && a[i] != x)   /* önce dizinin bitip bitmediğine bakılır */
        i++;

    if (i == n)                  /* döngüden neden çıkıldığı kontrol edilir */
        return -1;
    return i;
}

int main(void)
{
    int a[] = {4, 5, 2, 1, 7, 9};
    int n = 6;
    int aranan[] = {7, 5, 11};
    int k, yer;

    for (k = 0; k < 3; k++) {
        yer = dogrusal_ara(a, n, aranan[k]);
        if (yer == -1)
            printf("x = %d: dizide yok\n", aranan[k]);
        else
            printf("x = %d: %d. indiste\n", aranan[k], yer);
    }
    return 0;
}
```

Çıktı:

```
x = 7: 4. indiste
x = 5: 1. indiste
```

```
x = 11: dizide yok
```

4.6 Bu Örnekten Çıkan Üç Kural

- Adım sayısı belli olmayan bir iş için `while` türü bir döngü kullanın. İş bitince kendiliğinden biten bir döngü kurun.
- Dizi bitebiliyorsa önce bitip bitmediğini kontrol edin.
- Döngüden çıkınca mesaj vermeden önce neden çıktığınızı kontrol edin.

5. Bir Algoritma Nasıl Tasarlanır?

Hoca, birinci sınıftan beri söylenen tasarım sürecini yeniden özetledi:

1. **Problemi anlayın.** Ne isteniyor? Girişler ne, çıkışlar ne? Bu girişten bu çıkışa gitmek için hangi işler yapılmalı?
2. **Elde küçük bir örnek çözün.** Uç noktalarda ne oluyor, şart sağlanırsa ve sağlanmazsa ne yapılmalı, bunlara bakın.
3. **Algoritmayı tasarlayın.** Yarı kod (pseudocode) veya akış diyagramı ile taslağını çıkarın.
4. **Doğruluğundan emin olun.** Probleme göre gerekli durumları deneyin: genel durum, ilk ve son eleman gibi uç noktalar, elemanın dizide olmadığı gibi özel durumlar.
5. **Analiz edin.** Algoritmayı zaman ve yer açısından değerlendirin.
6. **Sorun varsa geri dönün.** Analiz sıkıntılı çıktıysa problemi anlamaya ya da tasarıma dönün ve baştan toparlayın.
7. **Ancak analizden memnunsanız kodlayın.**

Kodlamanın sona bırakılmasının nedeni yamadır. Her şey kodlandıktan sonra hata bulunursa genellikle yama yapılır: “oraya onu, buraya bunu ekleyeyim.” Program bir süre sonra gereksiz yamalarla dolu hale gelir. Yarı kod veya akış diyagramı aşamasında ise yama yapmadan düzgün bir tasarıma varmak çok daha kolaydır.

6. İyi Bir Algoritmanın Ölçütleri

Artık algoritmanın çalışması yetmez, tasarım olarak iyi olması da istenir. Algoritma analizi bu yüzden Veri Yapıları ve Algoritma Analizi derslerinin odağındadır. Hoca üç ölçüt saydı.

6.1 Verimlilik

Algoritma önce zamana, sonra yere göre verimli olmalıdır. Hoca bunun tarihini kendi öğrenciliğinden anlattı. O dönemde donanımlar çok yetersizdi. 256K belleği olan, sabit diski olmayan bilgisayarlarda çalışılır, her şey disketlerde tutulurdu. Bir diskette işletim sistemi, ikinci diskette yapılacak işler olurdu. Bu şartlarda hem zamanı hem yeri çok iyi kullanan algoritmalar tasarlamaya çalışılırdı.

Donanım gelişince bir rahatlama başladı: “Nasıl yazarsam yazayım hızlı çalışır, yer de bol.” Bugün ise verimlilik iki nedenle yeniden önemlidir:

- **Gömülü sistemler (embedded systems):** Çok sayıda algoritma gömülü sistemlerde geliştiriliyor. Bazı gömülü sistemlerin işletim sistemi daha basit, imkanları daha kısıtlıdır. Hoca, cep telefonunun da bir gömülü sistem olduğunu söyledi.
- **Derin öğrenme (deep learning):** Yapay sinir ağlarının simülasyonu 1940’lardan beri vardı, ama donanım yetmediği için çok temel seviyede kullanılıyordu. Bugün bir milyon örnekle eğitilen, 6 gün hatta bir ay çalışan programlar var. Donanım ve yer sonuna kadar kullanılıyor. Bir de siz kötü algoritma tasarlarsanız iş iyice zorlaşır.

6.2 Basitlik

Tasarım olabildiğince basit olmalıdır. Az önceki döngüyü kırma örneği basitlik açısından da yanlıştır, çünkü okuyanın kafasını karıştırır: “Ne oldu da döngüyü kırdı acaba?” Bu kadar basit bir algoritmada anlaşılabilir, ama karmaşık bir algoritmada sizden sonra kodu kullanacak kişinin de kolayca anlayabileceği bir yapı gerekir.

6.3 Genellik

Genel bir algoritma tasarlanmalıdır (generality). İlk sınıflarda yalnız verilen dizi için çalışan algoritmalar yazılır. Öğrenme arttıkça bu sorun azalır, ama yalnız bazı şartları sağlayan algoritmalar yine olmamalıdır. Hocanın örneği metin aramadır: Algoritma yalnız küçük harfler için değil, örneğin ASCII tablosundaki bütün karakterler için doğru çalışmalıdır.

7. Eratosthenes Kalburu ile Asal Sayıların Bulunması

Problem şudur: Verilen bir n için 2'den n'ye kadar olan bütün asal sayıları bulun. Hocanın örneğinde $n = 25$'tir. Bunun bilinen çözümü Eratosthenes kalburudur (Sieve of Eratosthenes). Asal olmayanları eleye eleye gittiği için "kalbur" adını alır.

> **Ders sonrası not:** Hoca yöntemin M.Ö. 200 civarından kaldığını, emin olmadığını belirterek söyledi. Eratosthenes büyük olasılıkla yaklaşık M.Ö. 276-194 yılları arasında yaşamıştır; söylenen tarih bu aralıkla uyumludur.

Kendisinden ve 1'den başka bir sayıya bölünemeyen sayılara asal sayı denir. Bilinen en küçük asal 2'dir, başlangıç noktası budur.

7.1 İlk Fikir: Bölünüyor mu Diye Bakmak

Soru: Bu problemi nasıl çözersiniz? **Cevap:** Bir öğrenci her sayının kendinden önceki sayılara bölünüp bölünmediğini incelemeyi önerdi. 3 için 2'ye, 4 için 2 ve 3'e, 5 için 2, 3 ve 4'e bakılır. Öğrenci ayrıca önce (2 hariç) sayının çift mi tek mi olduğuna bakılabileceğini ekledi. Hoca bunun doğru çalışacağını söyledi ve daha verimli yapılıp yapılamayacağını sordu.

İlk iyileştirme, ilk bölen bulununca çıkmaktır. Hoca ardından asıl soruyu sordu: 11 için 2, 3 ve 5'e bölünmüyorsa 7'ye bakmak gerekir mi? Bir öğrenci karekökü geçince bakmaya gerek olmadığını söyledi. Hocanın gerekçesi şudur: 14 için çarpanlardan biri 7'dir, ama diğer çarpan 2'dir ve 7'den küçüktür. Yani 14'e 2 üzerinden zaten uğranmıştır. Aynı şekilde 6 için 3'e, 8 için 4'e bakmaya gerek yoktur, çünkü 2 oraya uğramıştır.

> **Ders sonrası not:** Hoca, karekökü geçtikten sonra 7'nin katı olmaya imkan olmadığını söyledi; bu ifade tam değil, 14 hem 7'nin katıdır hem de 7, $\sqrt{14} \approx 3{,}74$ değerinden büyüktür. Doğrusu şudur: $m = a \cdot b$ ve $a \leq b$ ise $a \leq \sqrt{m}$ olur. Yani bileşik bir sayının karekökten büyük olmayan bir böleni mutlaka vardır, bakmak için karekök yeterlidir. 11 için $\sqrt{11} \approx 3{,}32$ olduğundan derste bakılan 5 de gereksizdir; 2 ve 3 yeter.

Karekök sınırıyla bu yöntem şöyle işler: 5 ve 7 için yalnız 2'ye bakılır, 9 için 2 ve 3'e bakılır, 3'e bölündüğü için 9 elenir.

> **Ders sonrası not:** Hoca karekök 5 için 2, karekök 7 için "2,5 falan" dedi; doğrusu $\sqrt{5} \approx 2{,}24$ ve $\sqrt{7} \approx 2{,}65$'tir. Hoca ayrıca "2'nin katı değilse 3'ün de katı değildir" dedi. Bu genel olarak yanlıştır (9 tek sayıdır ve 3'ün katıdır). 5 için 3'e bakılmamasının nedeni $3 > \sqrt{5}$ olmasıdır.

7.2 Kalbur Fikri: Katları Elemek

Hoca soruyu tersine çevirdi: "Bu sayı kimlere bölünüyor?" diye bakmak yerine "Bu sayının katları kim?" diye elesek daha kolay olmaz mı? "9, 3'e bölünüyor mu?" diye kontrol etmek yerine 3'ün katlarını eleyince o kontrolden kurtuluruz.

Soru: İkiyle eledikten sonra yeni bir dizi mi oluşturuyoruz, yoksa orijinal diziye mi bakıyoruz? **Cevap:** Tek bir dizi yeterlidir. Başlangıçta hepsine asal muamelesi yapılır ve 1 yazılır. Asal olmadığı anlaşılanlar 0 yapılır.

Adım adım $n = 25$ için:

Adım	Elenen sayılar	Açıklama
2'nin katları	4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24	Hiçbir şey kontrol etmeden yarısından kurtulunur.
$p = 3$	9, 12, 15, 18, 21, 24	Elemeye $3^2 = 9$'dan başlanır, 3'er artılır.
$p = 4$	yok	4 elenmiştir, katlarına bakılmaz.
$p = 5$	25	Elemeye $5^2 = 25$'ten başlanır.
Dur	yok	$\sqrt{25} = 5$, son bakılan sayı 5'tir.

Kalan asal sayılar: 2, 3, 5, 7, 11, 13, 17, 19, 23.

Bu tabloda iki noktaya dikkat etmek gerekir.

Birincisi, 3'ün katları arasında 12, 18 ve 24 de çift sayıdır ve zaten elenmişlerdir. Ancak bir öğrenci "9'dan sonra 15 gider" deyince hoca şunu belirtti: Bunu atlamak için `if` ile kontrol etmek gerekir. Hiç bakmadan 3'er 3'er ekleyip gitmek daha ucuzdur.

İkincisi, bir sayının katlarını elemeden önce o sayının hâlâ asal işaretli olup olmadığına bakmak zorunludur. Bu kontrol yapılmazsa örneğin 9'un katları da boşuna elenmeye çalışılır.

7.3 Neden 7'nin Katlarına Bakılmaz?

$n = 25$ için son olarak 5'in katları elenir. Hocanın gerekçesi şöyledir: 7'nin katları 14, 21, 28, 35 diye gider. 14'e çarpanı olan 2, 21'e 3, 35'e 5 zaten uğramıştır. 7'nin kendi işe yarayabileceği ilk kat $7^2 = 49$'dur ve 25'ten büyüktür.

> **Ders sonrası not:** Hoca "28'e 4 uğradı" dedi; doğrusu 4 elenmiş olduğu için katlarına bakılmaz, 28'i 2 eler.

7.4 Algoritmanın Tasarımı

Hoca aradan sonra akış diyagramını kurdu. 0, 1 ve 2 numaralı gözlere 1 yazılır. Amaç 0 ve 1'in asal olup olmadığını tartışmak değildir, bu gözlerde rastgele değer kalmamasıdır. Sayılar yazdırılırken 2'den başlanacağı için bu gözler sonuca girmez.

Sonra 4'ten başlayıp 2'şer artarak n'ye kadar çift gözler 0 yapılır. 3'ten başlayıp 2'şer artarak tek gözler 1 yapılır. Bunlar şu an için asal kabul edilen sayılardır. Sonra karekök hesaplanır. Hoca burada kaçak olmaması için yukarı doğru yuvarlamanın gerekebileceğini söyledi. Ana döngü 3'ten karekök sınırına kadar 2'şer artar.

```
n oku
A[0] = A[1] = A[2] = 1
p = 4'ten n'e kadar 2'şer:      A[p] = 0
p = 3'ten n'e kadar 2'şer:      A[p] = 1
nn = sqrt(n) + 1
p = 3'ten nn'e kadar 2'şer:
    eğer A[p] != 0 ise:
        j = p * p
        j <= n olduğu sürece:
            A[j] = 0
            j = j + p
p = 2'den n'e kadar:
    eğer A[p] == 1 ise p yazdır
```

> **Ders sonrası not:** 01:06 civarında altyazıda "A[p] sıfıra eşitse katlarını eleyeceksiniz" geçiyor; doğrusu tasarımdaki gibi $A[p] \neq 0$ ise katlar elenir. $A[p] = 0$ ise bir sonraki sayıya geçilir. Hoca bir önceki cümlede bunu doğru söylüyor.

Aşağıdaki C kodu akış diyagramının karşılığıdır. `gcc -Wall` ile derlenmiş (`-lm` gerekir), n için 2-399 arası tüm değerlerde kaba kuvvet ile aynı sonucu verdiği denenmiştir. A dizisi 0'dan n'ye kadar indislendiği için $n+1$ gözlü olmalıdır.

```c
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

int main(void)
{
    int n, nn, p, j;
    int *A;

    if (scanf("%d", &n) != 1 || n < 2) {
        printf("n en az 2 olmalidir\n");
        return 1;
    }
    A = malloc((n + 1) * sizeof(int));   /* 0..n indisleri için n + 1 göz */
    if (A == NULL)
        return 1;

    A[0] = A[1] = A[2] = 1;              /* içlerinde rastgele değer kalmasın */
    for (p = 4; p <= n; p += 2)          /* 2'nin katları kontrolsüz elenir */
        A[p] = 0;
    for (p = 3; p <= n; p += 2)          /* tek sayılar başlangıçta asal sayılır */
        A[p] = 1;

    nn = (int)sqrt(n) + 1;               /* karekökte kaçak olmasın diye */
    for (p = 3; p <= nn; p += 2) {
        if (A[p] != 0) {                 /* yalnız asalın katları elenir */
            j = p * p;                   /* elemeye karesinden başlanır */
            while (j <= n) {
                A[j] = 0;
                j += p;
            }
        }
    }

    for (p = 2; p <= n; p++)
        if (A[p] == 1)
            printf("%d ", p);
    printf("\n");

    free(A);
    return 0;
}
```

$n = 25$ girişi için çıktı: `2 3 5 7 11 13 17 19 23`. Burada $nn = 6$ olur, ana döngü yalnız $p = 3$ ve $p = 5$ için çalışır.

7.5 Üç İyileştirme

Hoca sonunda, atladığını söylediği "$j = p^2$" iyileştirmesine döndü. Önce iyileştirilmemiş hali gösterdi: 3'ün katlarından başlansaydı 6, 9, 12, 15, 18, 21, 24 elenirdi. 6'ya 2 zaten uğramıştır. 5 için 10'a 2, 15'e 3, 20'ye 2 uğramıştır. O halde 3 için 9'dan, 5 için doğrudan 25'ten başlanabilir. Üç iyileştirme şunlardır:

- 2'nin katlarını kontrol etmeden baştan elemek. 100 elemanın 50'si 2'nin katıdır.
- Yalnız $\sqrt{n}$'ye kadar olan sayıların katlarını elemek.
- Asal p için elemeye p^2'den başlamak.

Bunlar yapılmasa da algoritma doğru çalışır. 5'ten sonra 7'nin, 11'in katları da elense sonuç değişmez, ama örneğin 22'ye 2 zaten uğramıştır. İyileştirmeler yalnız gereksiz işi azaltır. $n = 25$ için dizi üzerindeki sıfırlama sayısı şöyledir (ara adım eklendi):

Tasarım	$p=2$	$p=3$	$p=5$	$p=7$	$p=11$	Toplam
Her asal için $2p$'den başla	11	7	4	2	1	25
Karekökte dur	11	7	4			22
Karekökte dur, p^2'den başla	11	6	1			18

2 ile 25 arasında 15 bileşik sayı vardır. Bu yüzden son satırda yalnız 3 tekrar kalır: 12, 18 ve 24.

> **Ek bilgi (derste anlatılmadı):** Kalburun zaman karmaşıklığı $O(n \log \log n)$'dir. Bu, her asal p için yaklaşık n/p eleme yapılması ve asalların terslerinin toplamının $\log \log n$ gibi büyümesinden gelir. Ayrıca yuvarlamaya gerek olmadığı gösterilebilir: p tam sayı olduğundan $p \le \sqrt{n}$ ile $p \le \lfloor \sqrt{n} \rfloor$ aynı şeydir. Kayan nokta hatası düşünülüyorsa `p * p <= n` koşulu da kullanılabilir.
>
> **Hocanın vurgusu:** Bir algoritmayla ilk kez karşılaştığınızda hemen şahane bir çözüm üretmeniz beklenmez. Bu derslerde gördüğünüz algoritmalar size ilham vermeli, görüş açınızı genişletmelidir. Algoritma asla ezberlenmez. Farklı tarzları, yaratıcı fikirleri öğrenip uygulamaya çalışın.

8. İki Sıralı Dizinin Birleştirilmesi

Bu da birinci sınıfta görülmüş bir problemdir. Hoca bunu uç durumlara dikkat ederek yeniden çözdü. Örnek diziler şunlardır:

$$A = \{3,\ 5,\ 7,\ 10,\ 12,\ 16\}, \quad n_a = 6 \qquad\qquad B = \{1,\ 4,\ 18,\ 21\}, \quad n_b = 4$$

İkisi de sıralıdır. Bunlardan sıralı, 10 elemanlı yeni bir C dizisi elde edilmek istenir (merge).

8.1 İlk Akla Gelen Çözüm

İlk akla gelebilecek şey, iki diziyi arka arkaya koyup herhangi bir sıralama algoritmasını (selection sort, insertion sort, bubble sort, merge sort) çağırmaktır. Bu doğrudur ve çalışır, ama verimli değildir. Hocanın gerekçesi şudur: Sıralama algoritmalarının en iyisinin karmaşıklığı $n \log n$'dir. Insertion sort en iyi durumda n adımda biter, ama burası en iyi durum değildir.

> **Ders sonrası not:** Hoca "sıralama algoritmalarının en iyisi $n \log n$" dedi; bu alt sınır karşılaştırmaya dayalı sıralama (comparison sort) içindir. Dönem planındaki sayarak sıralama gibi yöntemler bu sınıfa girmez.

Oysa iki dizi sıralıdır ve bundan faydalanılmalıdır. 1 ile 3 karşılaştırıldığında 1, 3'ten küçükse A'nın geri kalan elemanları olan 5, 7, 10, 12, 16'dan da küçüktür.

> **Hocanın vurgusu:** Algoritma tasarlarken problemdeki durumdan faydalanabilir miyim diye sorun. Sırasız dizide eleman ararken yapacak bir şey yoktu, tek tek bakılıyordu. Burada iki sıralı dizi verildiğine göre bundan faydalanılmalıdır.

8.2 İki Göstergeli Çözüm

Bir öğrencinin önerdiği yöntem de budur: İlk elemanları karşılaştır, küçüğü yeni diziye yaz, yalnız o dizinin indisini artır. Bir dizi bitince diğerinin kalanını karşılaştırmadan kopyala. Her dizi için bir gösterge (pointer) tutulur. Hocanın örneğinde adımlar şöyledir:

Karşılaştırma	Küçük mü?	C'ye yazılan	i_a	i_b	i_c
$3 < 1$	Hayır	1	0	1	1

Karşılaştırma	Küçük mü?	C'ye yazılan	i_a	i_b	i_c
$3 < 4$	Evet	3	1	1	2
$5 < 4$	Hayır	4	1	2	3
$5 < 18$	Evet	5	2	2	4
$7 < 18$	Evet	7	3	2	5
$10 < 18$	Evet	10	4	2	6
$12 < 18$	Evet	12	5	2	7
$16 < 18$	Evet	16	6	2	8

Burada A biter. B'nin kalanı 18 ve 21 karşılaştırılmadan taşınır:

$$C = \{1,\ 3,\ 4,\ 5,\ 7,\ 10,\ 12,\ 16,\ 18,\ 21\}$$

Hoca bu örneği bilerek seçti. B daha kısa olduğu halde önce A bitti, çünkü B'nin sonunda daha büyük sayılar var.

> **Hocanın vurgusu:** En çok yapılan hatalardan biri, kısa olan dizinin önce biteceğini sanmaktır. Dizinin uzunluğu hangisinin önce biteceğini belirlemez.

8.3 Durma Şartı

Adım sayısı belli değildir, o halde bir `while` döngüsü gerekir. Dizilerden biri bittiği anda durulmalıdır. Yani $i_a < n_a$ **ve** $i_b < n_b$ olduğu sürece dönülür.

> **Hocanın vurgusu:** Sık yapılan hatalardan biri bağlaç olarak `||` (veya) kullanmaktır. `||` "şartlardan biri sağlanıyorsa dönmeye devam et" demektir. İstenen ise "biri bozulursa dışarı çık"tır.

Hoca durma şartını gereksiz yere karmaşıklaştıran iki yaklaşımı da eleştirdi. Birincisi, önce son elemanlara bakıp hangi dizinin önce biteceğine göre ayrı döngüler kurmaktır. İkincisi, dizinin sonuna sahte büyük bir sayı (gözcü, sentinel), örneğin en büyük tam sayıyı eklemektir. Böylece o dizi mutlaka geç biter. Hocanın yorumu: "Niye uğraşıyorsun?" Hangi dizi önce biterse çıkılır, yola devam edilir.

> **Ders sonrası not:** Hoca en büyük tam sayı olarak 32.767 dedi; bu 16 bitlik `int` içindir. Burada kullanılan `gcc` ile `int` 4 bayttır ve `INT_MAX` 2.147.483.647'dir. Doğru değer `<limits.h>` içindeki `INT_MAX` ile alınır.

Döngüden çıkınca yine neden çıkıldığına bakılır. $i_a < n_a$ ise B bitmiştir, A'nın kalanı taşınır. Değilse A bitmiştir, B'nin kalanı taşınır.

```
na, A[na], nb, B[nb] oku
ia = 0, ib = 0, ic = 0
(ia < na) ve (ib < nb) olduğu sürece:
    eğer A[ia] < B[ib] ise:  C[ic++] = A[ia++]
    değilse:                 C[ic++] = B[ib++]
eğer ia < na ise:
    i = ia'dan na-1'e kadar: C[ic++] = A[i]
değilse:
    i = ib'den nb-1'e kadar: C[ic++] = B[i]
C dizisini 0'dan ic-1'e kadar yazdır
```

Soru: Önce kaydedip sonra indisi artırmak gerekmiyor muydu? `ia++` yazıp doğrudan `C`'nin içine attık. **Cevap:** Son ek `++` zaten önce atar, sonra artırır. Açık hali şudur: `C[ic] = A[ia]; ia++; ic++;`

8.4 Küçük Bir Örnekle Tekrar

Hoca mantığı ikinci bir örnekle gösterdi: $A = \{1, 3, 5, 8\}$, $n_a = 4$ ve $B = \{2, 4, 10\}$, $n_b = 3$.

Karşılaştırma	Küçük mü?	C'ye yazılan	i_a	i_b	i_c
$1 < 2$	Evet	1	1	0	1
$3 < 2$	Hayır	2	1	1	2
$3 < 4$	Evet	3	2	1	3
$5 < 4$	Hayır	4	2	2	4
$5 < 10$	Evet	5	3	2	5
$8 < 10$	Evet	8	4	2	6

A biter, döngüden çıkılır. $i_a < n_a$ yanlış olduğu için B'nin kalanı olan 10 taşınır (ara adım eklendi). Sonuç $C = \{1, 2, 3, 4, 5, 8, 10\}$ olur.

8.5 Eşit Elemanlar

Soru: Elemanlardan eşit olanlar olsaydı dikkat edeceğimiz bir nokta olur muydu? **Cevap:** Kontrol küçüktür ile yapıldığı için eşitlikte diğer taraftaki eleman taşınır. A'da bir 3, B'de 3 ve 5 olsun. $3 < 3$ yanlış olduğundan B'deki 3 taşınır. B'nin göstergesi 5'e gelir. $3 < 5$ doğru olduğundan A'daki 3 taşınır. İki eleman arka arkaya gelir. Birleştirmede eşitse birini atmak gibi bir durum yoktur, bir sayı birden fazla kez geçebilir.

> **Ek bilgi (derste anlatılmadı):** Kod `A[ia] < B[ib]` ile yazıldığında eşitlikte önce B'deki eleman gelir. Koşul `A[ia] <= B[ib]` yapılırsa önce A'daki gelir ve eşit elemanların göreli sırası korunur. Bu özelliğe kararlılık (stability) denir. Sayılardan oluşan bu problemde sonuç değişmez, ancak kayıt sıralarken önem kazanır.

Aşağıdaki C kodu akış diyagramının karşılığıdır, `gcc -Wall` ile derlenip derste geçen üç örnekle çalıştırılmıştır.

```c
#include <stdio.h>

/* Sıralı A ve B dizilerini sıralı C dizisinde birleştirir.
   C'ye yazılan eleman sayısını döndürür. */
int birlestir(const int A[], int na, const int B[], int nb, int C[])
{
    int ia = 0, ib = 0, ic = 0, i;

    while (ia < na && ib < nb) {     /* dizilerden biri bitince çıkılır */
        if (A[ia] < B[ib])
            C[ic++] = A[ia++];
        else
            C[ic++] = B[ib++];
    }

    if (ia < na)                     /* B bitmiş: A'nın kalanı taşınır */
        for (i = ia; i < na; i++)
            C[ic++] = A[i];
    else                             /* A bitmiş: B'nin kalanı taşınır */
        for (i = ib; i < nb; i++)
            C[ic++] = B[i];

    return ic;
}

void yazdir(const int C[], int nc)
{
    int i;

    for (i = 0; i < nc; i++)
        printf("%d ", C[i]);
    printf("\n");
```

```
}

int main(void)
{
    int A1[] = {3, 5, 7, 10, 12, 16}, B1[] = {1, 4, 18, 21}, C1[10];
    int A2[] = {1, 3, 5, 8}, B2[] = {2, 4, 10}, C2[7];
    int A3[] = {3}, B3[] = {3, 5}, C3[3];

    yazdir(C1, birlestir(A1, 6, B1, 4, C1));
    yazdir(C2, birlestir(A2, 4, B2, 3, C2));
    yazdir(C3, birlestir(A3, 1, B3, 2, C3));
    return 0;
}
```

Çıktı:

```
1 3 4 5 7 10 12 16 18 21
1 2 3 4 5 8 10
3 3 5
```

8.6 Adım Sayısı

Hoca, birleştirip sıralamanın $n \log n$ olduğunu hatırlatıp bu çözümün kaç adım sürdüğünü sordu. Bir öğrenci "$m + n$" dedi. Hoca önce hangi dizinin önce biteceğinin son elemanlara bağlı olduğunu açıkladı: Son elemanı küçük olan dizi önce biter. Ya biri biter ya öbürü, kalan da taşınır. Toplamda en kötü durumda $n_a + n_b$ adım yapılır. Bu yüzden iki diziyi birleştirip sıralamaktan daha verimlidir.

> **Ders sonrası not:** Hoca "A'nın son elemanı büyükse B biter, n_b kez dolaşılır" dedi; doğrusu ilk döngü n_b kez değil, n_b artı A'dan taşınan eleman sayısı kadar döner. Örneğimizde A bittiği için $n_a + 2 = 8$ dönüş ve 2 kopyalama vardır. Toplam taşıma ise "en fazla" değil, her zaman tam $n_a + n_b$'dir, çünkü her eleman tam bir kez taşınır. Karşılaştırma sayısı $\min(n_a, n_b)$ ile $n_a + n_b - 1$ arasındadır. Hocanın en kötü diye örneklediği "A'nın tamamı taşındı, B'nin tamamı kaldı" durumunda yalnız n_a karşılaştırma yapılır, yani karşılaştırma açısından en kötü durum bu değildir. Bu sınırlar 20.000 rastgele dizi çiftiyle python'da doğrulanmıştır.

9. Düşünme Sorusu: Hash Tablosunda Ortalama Yerleşme Adımı

Hoca dersin sonunda bir soru bıraktı. Soru geçen dönem Algoritma Analizi dersinde sorulmuştu. Hocaya göre neredeyse herkes çözmüş, ama iki üç kişi dışında verimli çözen olmamıştı. Hoca "Siz şu anda önyargısız bakabiliyorsunuz, basit mi çözeceksiniz karışık mı?" diye sordu ve bir sonraki derse kadar düşünülmesini istedi.

Kurulum şudur: $N = 10$ gözlü, başlangıçta sıfırlanmış bir dizi vardır (örneğin `calloc` ile). Bir x sayısı x mod 10 numaralı göze yerleşmek ister. Örneğin 3 üçüncü, 2 ikinci, 45 beşinci göze gider. Göz doluysa boş göz bulana kadar sırayla ilerlenir, sondan sonra başa dönülür. Bu, hashing yöntemlerinden biridir ve bir A dizisindeki elemanlar sırayla bir B dizisine yerleştirilir. A'nın eleman sayısı B'nin göz sayısından azdır, genelde yaklaşık %50'si kadardır. Tablo zaten dolsun istenmez.

> **Ders sonrası not:** Altyazıda hoca 19 için "19 mod 19" diyor; doğrusu $19 \bmod 10 = 9$'dur.

Sorulan şudur: Elemanlar ortalama kaçıncı adımda yerleşmiştir? Her elemanın kaçıncı denemede yerleştiği bulunur ve ortalaması alınır. Herkes ilk seferde yerleşirse ortalama 1 olur.

Hocanın örneğinde sıra 2, 3, 22, 9, 19'dur:

x	x mod 10	Denenen gözler	Yerleştiği göz	Deneme sayısı
2	2	2	2	1
3	3	3	3	1

x	x mod 10	Denenen gözler	Yerleştiği göz	Deneme sayısı
22	2	2 (dolu), 3 (dolu), 4	4	3
9	9	9	9	1
19	9	9 (dolu), 0	0	2

Hoca adım sayılarını verdi, ortalamayı bıraktı. Ortalama şöyledir (ara adım eklendi):

$$\frac{1+1+3+1+2}{5} = \frac{8}{5} = 1{,}6$$

Hoca son olarak aynı göze yerleşmek isteyen elemanların da olabileceğini söyledi: 44, 54, 74, 84 elemanlarının hepsi 4. gözü ister. Bu dört eleman sırasıyla 4, 5, 6 ve 7. gözlere 1, 2, 3 ve 4. denemede yerleşir. Ortalama $(1+2+3+4)/4 = 2{,}5$ olur (ara adım eklendi). Sorunun verimli çözümü derste verilmedi.

> **Ek bilgi (derste anlatılmadı):** Bu yöntemin adı doğrusal yoklamadır (linear probing). Dolu eleman sayısının göz sayısına oranına doluluk oranı (load factor) denir; hocanın söylediği %50 bu orandır. Boş gözü 0 ile göstermek, tabloya $x = 0$ gelebiliyorsa sorun çıkarır, çünkü dolu göz boş sanılır.

Dersin Toparlaması

Ders, çok bilinen basit problemler üzerinden tasarım alışkanlığı kazandırmayı amaçladı. Doğrusal arama adım sayısı belli olmayan işler için `while` kullanmayı, dizinin bitişini önce kontrol etmeyi ve döngüden çıkınca nedenine bakmayı öğretti. Tasarım süreci problemi anlamakla başlar, örnek çözmek, tasarlamak, doğrulamak ve analiz etmekle sürer, kodlama en sona kalır. İyi bir algoritma verimli, basit ve geneldir.

Eratosthenes kalburu, “kime bölünüyor?” sorusunu “katları kim?” sorusuna çevirmenin ve küçük iyileştirmelerin gereksiz işi nasıl azalttığını gösterdi. Sıralı dizilerin birleştirilmesi, problemdeki avantajdan yararlanmanın $n \log n$ yerine doğrusal bir çözüm verdiğini ve durma şartının sade kurulması gerektiğini gösterdi. Dersin sonunda hash tablosunda ortalama yerleşme adımı sorusu bir sonraki derste konuşulmak üzere bırakıldı.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.